Parallel & Distributed Computing Lab

IMSciences

Lab Instructor: Saim Sanan

Week 4, Lab 2

Matrix Multiplication with OpenMP

---

1 Ideas

- **Matrix multiply** — $C = A \times B$. Size $N = 500$. Each $C[i][j]$ is a dot product of row i of A and column j of B.
- Every A and B cell is 1, so each $C[i][j]$ equals $N = 500$. Check `C[0][0]`.
- Three loops: `i`, `j`, `k`. Only the **outer** `i` loop is split. Each thread gets different rows of C. No `reduction`. No `atomic`.
- `private(j, k)` — each thread has its own `j` and `k`. `i` is already split by `parallel for`.
- Time with `omp_get_wtime()` on both programs. Do not use `clock()`. Init of A, B, C is outside the stopwatch.

`C[0][0]` must be 500.
speedup $= T_{\text{serial}}/T_{\text{parallel}}$.

2 Commands

Terminal

```
nproc
nano matrix_serial.c
gcc -fopenmp matrix_serial.c -o mserial
./mserial

nano matrix_parallel.c
gcc -fopenmp matrix_parallel.c -o mparallel
./mparallel
```

`nproc` should be 2 or more, or parallel will not get faster.
nano: Ctrl+O, Enter (save), Ctrl+X (quit).
Forgot `./` → command not found.
Forgot `-fopenmp` → error on `omp.h`, or still one thread.
`static` on the arrays: $500 \times 500 \times 3$ doubles is too large for the small stack.

3 Serial (one worker)

C

```
#include <stdio.h>
#include <omp.h>

#define N 500

int main(void) {
    static double A[N][N];
    static double B[N][N];
```

```
    static double C[N][N];
    int i, j, k;
    double start, end;

    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++) {
            A[i][j] = 1.0;
            B[i][j] = 1.0;
            C[i][j] = 0.0;
        }
    }

    start = omp_get_wtime();
    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++) {
            for (k = 0; k < N; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
    end = omp_get_wtime();

    printf("C[0][0] = %f (expected %d)\n", C[0][0], N);
    printf("serial time = %f seconds\n", end - start);
    return 0;
}
```

`-fopenmp` is only for `wtime`. There is no `#pragma`, so this still uses one worker.

4 Parallel (split `i` only)

The new lines:

C

```
omp_set_num_threads(4);

#pragma omp parallel for private(j, k)
```

C

```
#include <stdio.h>
#include <omp.h>

#define N 500

int main(void) {
    static double A[N][N];
    static double B[N][N];
    static double C[N][N];
    int i, j, k;
    double start, end;

    omp_set_num_threads(4);

    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++) {
            A[i][j] = 1.0;
            B[i][j] = 1.0;
            C[i][j] = 0.0;
        }
    }
```

```
    start = omp_get_wtime();
    #pragma omp parallel for private(j, k)
    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++) {
            for (k = 0; k < N; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
    end = omp_get_wtime();

    printf("C[0][0] = %f (expected %d)\n", C[0][0], N);
    printf("parallel time = %f seconds\n", end - start);
    printf("threads = %d\n", omp_get_max_threads());
    return 0;
}
```

`C[0][0]` must still be 500. Parallel time should be lower than serial if `nproc` is 2 or more.

$$\text{speedup} = \frac{\text{serial time}}{\text{parallel time}}$$

Four threads is not always 4× (Amdahl). Use the times from this run.